

Agentes

FCEFyN - UNC

- Autor/es: Ing. Sandro Mauricio Comerci (LIDeSIA - FCEFyN)

Agentes

¿Cómo un LLM puede pasar de responder texto a usar herramientas externas?

Para dar respuesta a esta pregunta necesitamos: conectar con un LLM, probar una llamada simple, definir herramientas, diseñar un prompt ReAct, implementar el bucle `Thought → Action → Observation → Final Answer` y ejecutar ejemplos.

```
import warnings
warnings.filterwarnings(
    "ignore",
    message="datetime.datetime.utcnow"
)
warnings.filterwarnings("ignore", category=DeprecationWarning)
!pip install ddgs
```

Show hidden output

2. Imports y configuración inicial

Antes de crear el agente, importamos librerías generales y definimos tipos auxiliares.

```
import os
import re
import ast
import math
```

```
import operator
from dataclasses import dataclass
from datetime import datetime
from getpass import getpass
from typing import Callable, Dict, List, Optional, Tuple
```

3. Conexión con el proveedor del LLM

Un modelo de lenguaje (LLM) normalmente no se ejecuta dentro de nuestra notebook.
Lo habitual es consumirlo mediante una API remota, de forma similar a cómo una aplicación consume un servicio web.

En este ejemplo utilizamos **Hugging Face Router**, que funciona como una capa intermedia para acceder a distintos modelos de lenguaje hospedados en infraestructura externa.

La notebook actúa como un cliente:

1. El usuario escribe una consulta.
2. Python construye un prompt.
3. Ese prompt se envía por internet a la API del proveedor.
4. El proveedor ejecuta el modelo en servidores con GPU.
5. El modelo genera una respuesta.
6. La respuesta vuelve a la notebook como texto.

El procesamiento pesado ocurre del lado del proveedor, no en Colab.
Por eso podemos utilizar modelos grandes aunque nuestra notebook no tenga una GPU potente.

La comunicación ocurre mediante requests HTTP.
Internamente, el cliente de OpenAI envía mensajes JSON a un endpoint remoto.
Hugging Face Router implementa una interfaz compatible con OpenAI, por eso podemos reutilizar el mismo cliente cambiando únicamente la URL base (`base_url`).

```
from openai import OpenAI

if "HF_TOKEN" not in os.environ:
    os.environ["HF_TOKEN"] = getpass("Ingresá tu HF_TOKEN: ")

client = OpenAI(
    base_url="https://router.huggingface.co/v1",
    api_key=os.environ["HF_TOKEN"],
)

print("Cliente inicializado correctamente.")
```

```
Ingresá tu HF_TOKEN: ··········
Cliente inicializado correctamente.
```

```
models = client.models.list()

print("Modelos disponibles:")
for m in models.data[:30]:
    print("-", m.id)
```

```
Modelos disponibles:
- deepseek-ai/DeepSeek-V4.1-Flash
- Qwen/Qwen3.8-27B
- meta-llama/Llama-3.1-8B-Instruct
- zai-org/GLM-5.3-Flash
- deepseek-ai/DeepSeek-R1
- moonshotai/Kimi-K3
- Qwen/Qwen3-8B
- deepseek-ai/DeepSeek-V4-Flash-Vision-Exp
- inclusionAI/Ling-3.0-flash-VL
- moonshotai/Kimi-K2-Instruct
- zai-org/GLM-5.3
- google/gemma-4-31B-it
- openai/gpt-oss-120b
- deepseek-ai/DeepSeek-V4-Flash-0731
- prism-ml/Ternary-Bonsai-27B-gguf
- meta-models/Muse-Glimmer-30B
- deepseek-ai/DeepSeek-V4-Flash
- openai/gpt-oss-20b
- Qwen/Qwen3.5-9B
- deepseek-ai/DeepSeek-V4-Pro
- google/gemma-4-26B-A4B-it
- Qwen/Qwen3.8-2.4T-A95B
- Qwen/Qwen3.6-35B-A3B
- inclusionAI/Ling-3.0-flash-Fin
- zai-org/GLM-5.2
- deepseek-ai/DeepSeek-V4-Pro-0813
- Qwen/Qwen3.6-27B
- google/gemma-3-12b-it
- Qwen/Qwen3-Coder-30B-A3B-Instruct
- meta-llama/Llama-3.3-70B-Instruct
```

¿Por qué usamos la librería de OpenAI si el modelo está en Hugging Face?

B = 1 billion = 1.000 millones

```
MODEL_ID = "Qwen/Qwen2.5-7B-Instruct"

response = client.chat.completions.create(
    model=MODEL_ID,
    messages=[{"role": "user", "content": "Explica en una oración qué es
    temperature=0.2,
)

```

```
print(response.choices[0].message.content)

```

```
Un agente de IA es un programa o sistema diseñado para tomar decisiones y rea
```

```
response
```

```
ChatCompletion(id='oiBCAHg-6Ng1vN-9f82fb8b3c6ffb02', choices=
[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content='Un agente de IA es un programa o
sistema diseñado para tomar decisiones y realizar tareas de manera autónoma
utilizando inteligencia artificial, aprendiendo de los datos y adaptándose a
nuevas situaciones.', refusal=None, role='assistant', annotations=None,
audio=None, function_call=None, tool_calls=[]), seed=3256071366684055000)],
created=1778185826, model='Qwen/Qwen2.5-7B-Instruct-Turbo',
object='chat.completion', service_tier=None, system_fingerprint=None,
usage=CompletionUsage(completion_tokens=44, prompt_tokens=43,
total_tokens=87, completion_tokens_details=None, prompt_tokens_details=None,
cached_tokens=0), prompt=[])
```

```
def call_llm(messages: List[Dict[str, str]], temperature: float = 0.2) -
    response = client.chat.completions.create(
        model=MODEL_ID,
        messages=messages,
        temperature=temperature,
    )
    return response.choices[0].message.content
```

¿Qué es una herramienta?

Una **tool** (herramienta) es una función externa que un agente de IA puede utilizar para interactuar o realizar tareas específicas.

Los modelos de lenguaje son muy buenos generando texto, razonando y siguiendo instrucciones, pero tienen limitaciones importantes:

- no pueden acceder directamente a internet,
- no conocen información posterior a su entrenamiento,
- no ejecutan código Python,
- no leen archivos automáticamente,
- y no interactúan con bases de datos o APIs por sí solos.

Por eso aparecen las herramientas.

Las tools permiten extender las capacidades del LLM conectándolo con funciones reales de Python.

Por ejemplo, un agente podría usar herramientas para:

- realizar cálculos matemáticos,
- buscar información actualizada,

- consultar la fecha y hora,
- leer archivos,
- consultar bases de datos,
- ejecutar código.

Idea importante

El LLM NO ejecuta herramientas directamente.

El modelo únicamente genera texto indicando qué acción quiere realizar.

Por ejemplo:

```
Action: calculator | 2 + 2
```

```
def tool_now(_: str = "") -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
from ddgs import DDGS

def web_search_tool(tool_input: str):

    query, params = parse_tool_input(tool_input)

    k = int(params.get("k", 5))

    with DDGS() as ddgs:

        results = list(
            ddgs.text(
                query,
                max_results=k
            )
        )

    if not results:
        return "No se encontraron resultados."

    output = []

    for r in results:

        title = r.get("title", "Sin título")
        url = r.get("href", "Sin URL")
        body = r.get("body", "Sin resumen")

        output.append(
            f"Título: {title}\n"
            f"URL: {url}\n"
            f"Resumen: {body}"
```

```
        )

    return "\n\n".join(output)

def parse_tool_input(text: str):
    """
    Permite recibir inputs tipo:
    'historia de la Argentina | k=5'
    """
    parts = [p.strip() for p in text.split("|")]

    query = parts[0]
    params = {}

    for part in parts[1:]:
        if "=" in part:
            key, value = part.split("=", 1)
            params[key.strip()] = value.strip()

    return query, params
```

En un agente ReAct, las herramientas necesitan almacenarse de forma estructurada.

Cada herramienta tiene:

- un nombre,
- una descripción,
- y una función asociada que será ejecutada por Python.

Para representar esa estructura utilizamos `@dataclass`.

```
@dataclass
class Tool:
    name: str
    description: str
    func: Callable[[str], str]


def build_tools_description(tools: Dict[str, Tool]) -> str:
    return "\n".join(
        f"- {tool.name}: {tool.description}"
        for tool in tools.values()
    )
```

```
TOOLS: Dict[str, Tool] = {

    "now": Tool(
        name="now",
        description=(
            "Devuelve la fecha y hora actual.\n\n"

            "Usar SOLO para:\n"
```

```
            "- fecha actual\n"
            "- hora actual\n"
            "- día actual\n\n"

            "NO usar para:\n"
            "- búsquedas web\n"
            "- conceptos\n"
            "- historia\n\n"

            "Formato:\n"
            "Action: now |\n\n"

            "Output:\n"
            "String con fecha y hora.\n\n"

            "Ejemplo:\n"
            "Action: now |"
        ),
        func=tool_now,
    ),

    "web_search": Tool(
        name="web_search",
        description=(
            "Busca información actualizada en internet.\n\n"

            "Usar para:\n"
            "- conceptos\n"
            "- investigación\n"
            "- noticias\n"
            "- historia\n"
            "- personas\n"
            "- tecnología\n"
            "- enlaces web\n\n"

            "NO usar para:\n"

            "- cálculos matemáticos\n\n"

            "Formato:\n"
            "Action: web_search | consulta | k=5\n\n"

            "Parámetros:\n"
            "- consulta: texto a buscar\n"
            "- k: cantidad máxima de resultados\n\n"

            "Output:\n"
            "Lista de resultados con título, URL y resumen.\n\n"

            "Ejemplos:\n"
            "Action: web_search | historia de Los Simpson | k=5\n"
            "Action: web_search | qué es LangChain | k=3\n"
            "Action: web_search | últimas noticias IA | k=10"
        ),
        func=web_search_tool,
```

```
    ),
}

TOOLS_DESC = build_tools_description(TOOLS)

print(TOOLS_DESC)
```

```
- now: Devuelve la fecha y hora actual.

Usar SOLO para:
- fecha actual
- hora actual
- día actual

NO usar para:
- búsquedas web
- conceptos
- historia

Formato:
Action: now |

Output:
String con fecha y hora.

Ejemplo:
Action: now |
- web_search: Busca información actualizada en internet.

Usar para:
- conceptos
- investigación
- noticias
- historia
- personas
- tecnología
- enlaces web

NO usar para:
- cálculos matemáticos

Formato:
Action: web_search | consulta | k=5

Parámetros:
- consulta: texto a buscar
- k: cantidad máxima de resultados

Output:
Lista de resultados con título, URL y resumen.

Ejemplos:
Action: web_search | historia de Los Simpson | k=5
Action: web_search | qué es LangChain | k=3
Action: web_search | últimas noticias IA | k=10
```

Prompt ReAct

ReAct significa **Reasoning + Acting**.

La idea principal es que el modelo no responda inmediatamente, sino que pueda:

1. razonar,
2. decidir una acción,
3. usar herramientas,
4. observar resultados,
5. y continuar razonando hasta llegar a una respuesta final.

En lugar de generar una única respuesta directa, el agente construye un ciclo iterativo de pensamiento y acción.

Estructura general

El formato típico de ReAct es:

```
Thought: razonamiento breve
Action: nombre_tool | input
Observation: resultado de la herramienta
FINAL: respuesta final al usuario
```

```
SYSTEM_PROMPT = f"""
Sos un agente ReAct.

Tu objetivo es:
1. Razonar paso a paso.
2. Elegir herramientas cuando sea necesario.
3. Observar resultados.
4. Llegar a una respuesta final.

================================================
HERRAMIENTAS DISPONIBLES
================================================

{TOOLS_DESC}

================================================
FORMATO OBLIGATORIO
================================================

Si necesitás usar una herramienta respondé EXACTAMENTE así:

Thought: razonamiento breve
Action: nombre_tool | input

```

```
25
26 Ejemplos válidos:
27
28 Thought: Necesito conocer la fecha actual.
29 Action: now |
30
31 Thought: Necesito buscar información sobre Los Simpson.
32 Action: web_search | historia de Los Simpson | k=5
33
34
35 ================================================
36 REGLAS IMPORTANTES
37 ================================================
38
39 1. SOLO podés usar estas herramientas:
40 {", ".join(TOOLS.keys())}
41
42 2. NO inventes herramientas.
43
44 3. NO inventes observaciones.
45
46 4. NUNCA escribas Observation.
47 La Observation será agregada automáticamente por Python.
48
49 5. Escribí SOLO UNA acción por respuesta.
50
51 6. El formato Action SIEMPRE debe contener "|".
52
53 7. Aunque una tool no necesite input, igualmente escribí:
54 Action: now |
55
56 8. Cuando tengas suficiente información respondé EXACTAMENTE:
57
58 8b. Solo utiliza una vez tu herramienta de busqueda por cada solicitud d
59
60 FINAL: respuesta final para el usuario
61
62 9. NO escribas:
63 - Final Answer:
64 - Respuesta Final:
65 - Answer:
66 - Resultado:
67
68 10. Si necesitás usar una herramienta:
69 NO respondas FINAL todavía.
70
71 11. Tus pensamientos deben ser cortos y claros.
72
73 12. No expliques el formato.
74 Solo seguí el formato.
75
76
77 ================================================
78 EJEMPLO COMPLETO
79 ================================================
```

```
Thought: Necesito saber la fecha actual.
Action: now |

Observation: 2026-05-07 14:30:00

Thought: Ya tengo la fecha actual.

FINAL: Hoy es 7 de mayo de 2026.
""".strip()

print(SYSTEM_PROMPT)

```

```
Sos un agente ReAct.

Tu objetivo es:
1. Razonar paso a paso.
2. Elegir herramientas cuando sea necesario.
3. Observar resultados.
4. Llegar a una respuesta final.

================================================
HERRAMIENTAS DISPONIBLES
================================================

- now: Devuelve la fecha y hora actual.

Usar SOLO para:
- fecha actual
- hora actual
- día actual

NO usar para:
- búsquedas web
- conceptos
- historia

Formato:
Action: now |

Output:
String con fecha y hora.

Ejemplo:
Action: now |
- web_search: Busca información actualizada en internet.

Usar para:
- conceptos
- investigación
- noticias
- historia
- personas
- tecnología
- enlaces web

NO usar para:
- cálculos matemáticos
```

```
    Formato:
    Action: web_search | consulta | k=5

    Parámetros:
    - consulta: texto a buscar
    - k: cantidad máxima de resultados

    Output:
    Lista de resultados con título, URL y resumen.

    Ejemplos:
```

Parser de acciones

El modelo devuelve texto plano.
Para que el agente pueda ejecutar herramientas reales, Python necesita interpretar ese texto automáticamente.

El parser busca líneas con formatos como:

```
Action: web_search | transformers
```

Si encuentra una acción válida:

- extrae el nombre de la herramienta,
- extrae el input,
- y ejecuta la función correspondiente.

También detecta cuándo el modelo terminó mediante patrones como:

```
Final Answer: ...
```

Esto permite transformar texto generado por el LLM en acciones reales ejecutadas por Python.

```
ACTION_RE = re.compile(r"^\s*Action:\s*([\w\-]+)\s*\|\s*(.+)\s*$", re.MU
FINAL_RE = re.compile(r"Final Answer:\s*(.*)", re.DOTALL)


def parse_action(llm_output: str) -> Optional[Tuple[str, str]]:
    match = ACTION_RE.search(llm_output)
    if not match:
        return None
    return match.group(1).strip(), match.group(2).strip()


def parse_final_answer(llm_output: str) -> Optional[str]:
    match = FINAL_RE.search(llm_output)
    if not match:
```

```
        return None
    return match.group(1).strip()

```

Bucle ReAct del agente

Este es el núcleo del agente ReAct.

El ciclo funciona de manera iterativa:

1. se envía la consulta al LLM,
2. el modelo razona,
3. decide si necesita usar una herramienta,
4. Python ejecuta la tool,
5. la observación vuelve al modelo,
6. y el proceso se repite hasta obtener una respuesta final.

```
Usuario
   ↓
LLM genera Thought + Action
   ↓
Python detecta la tool
   ↓
Python ejecuta la función real
   ↓
Se obtiene una Observation
   ↓
La Observation vuelve al LLM
   ↓
El LLM continúa razonando
```

El bucle termina cuando el modelo responde con:

```
Final Answer: ...
```

```
def run_agent(user_query: str, max_steps: int = 5) -> Tuple[str, List[di
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_query},
    ]
    trace: List[dict] = []

    for step in range(1, max_steps + 1):
        llm_output = call_llm(messages)
        messages.append({"role": "assistant", "content": llm_output})

```

```
        final_answer = parse_final_answer(llm_output)
        action = parse_action(llm_output)

        if final_answer and not action:
            trace.append({"step": step, "llm_output": llm_output, "actio
            return final_answer, trace

        if action:
            tool_name, tool_input = action
            if tool_name not in TOOLS:
                observation = f"Error: la herramienta '{tool_name}' no e
            else:
                observation = TOOLS[tool_name].func(tool_input)

            trace.append({
                "step": step,
                "llm_output": llm_output,
                "action": tool_name,
                "input": tool_input,
                "observation": observation,
            })

            messages.append({
                "role": "user",
                "content": f"Observation: {observation}\nContinuá. Si ya
            })
            continue

        trace.append({"step": step, "llm_output": llm_output, "action":
        return llm_output, trace

    return "No se obtuvo una respuesta final dentro del máximo de pasos.
```

```
def print_trace(trace: List[dict]) -> None:
    for row in trace:
        print(f"\n--- Paso {row['step']} ---")
        print(row["llm_output"])
        if row.get("action"):
            print("\n[Tool ejecutada]")
            print("Action:", row["action"])
            print("Input :", row["input"])
            print("Observation:")
            print(row["observation"])

```

```
answer, trace = run_agent("qué es una herramienta en un agente de IA")
print_trace(trace)
print("\nFINAL:\n", answer)
```

```
--- Paso 1 ---
Thought: Para explicar brevemente qué es una herramienta en un agente de IA,

Action: web_search | qué es una herramienta en un agente de IA | k=1
```

```
[Tool ejecutada]
Action: web_search
Input : qué es una herramienta en un agente de IA | k=1
Observation:
Título: Agentes de IA: Qué Son, Cómo Funcionan y Herramientas 2026
URL: https://www.aprender21.com/blog/agentes-ia-guia
Resumen: March 20, 2026 - En 2026, herramientas como n8n, Claude Computer Use

--- Paso 2 ---
Thought: La observación proporciona una definición clara de qué es una herram

Final Answer: Una herramienta en un agente de IA es una funcionalidad que per

FINAL:
 Una herramienta en un agente de IA es una funcionalidad que permite al agent
```

```
1 answer, trace = run_agent("Qué fecha es hoy?")
2 print_trace(trace)
3 print("\nFINAL:\n", answer)
```

```
--- Paso 1 ---
Thought: Necesito saber la fecha actual.
Action: now |

FINAL:
 Thought: Necesito saber la fecha actual.
Action: now |
```

```
1 answer, trace = run_agent("Buscá información sobre Los Simpson y dame 5
2 print_trace(trace)
3 print("\nFINAL:\n", answer)
```

```
--- Paso 1 ---
Thought: Necesito buscar información sobre Los Simpson y obtener 5 enlaces re
Action: web_search | historia de Los Simpson | k=5

[Tool ejecutada]
Action: web_search
Input : historia de Los Simpson | k=5
Observation:
Título: Caso O. J. Simpson - Wikipedia, la enciclopedia libre
URL: https://es.wikipedia.org/wiki/Caso_O._J._Simpson
Resumen: De acuerdo a la historia de la fiscalía, después de que Simpson term

Título: La Historia de los Simpsons en 2 minutos - Periodista Digital
URL: https://www.periodistadigital.com/cultura/20160910/historia-curiosidades
Resumen: Homer, Marge, Bart y Lisa nos han regalado las historias más rocambo

Título: 35 años de Los Simpson: un curioso corto de 88 segundos... - Infobae
URL: https://www.infobae.com/historias/2022/04/19/35-anos-de-los-simpson-un-c
Resumen: El 19 de abril de 1987 en el Tracey Ullman Show se estrenaban unos c

Título: 'Los Simpson' de aniversario: 25 años de curiosidades - FormulaTV
URL: https://www.formulatv.com/noticias/42554/las-25-curiosidades-los-simpson
```

```
Resumen: A lo largo de este cuarto de siglo de vida, la familia Simpson ha lo

Título: OJ Simpson es absuelto tras terminar su libertad condicional
URL: https://www.dw.com/es/oj-simpson-es-absuelto-tras-terminar-su-libertad-c
Resumen: Simpson, de 74 años, fue liberado de la cárcel de Nevada en 2017, tr

--- Paso 2 ---
Thought: Ya tengo 5 enlaces relacionados con la historia de Los Simpson.

Final Answer: Aquí te presento 5 enlaces relacionados con la historia de Los

1. [Caso O. J. Simpson](https://es.wikipedia.org/wiki/Caso_O._J._Simpson)
2. [La Historia de los Simpsons en 2 minutos](https://www.periodistadigital.c
3. [35 años de Los Simpson: un curioso corto de 88 segundos...](https://www.i
4. ['Los Simpson' de aniversario: 25 años de curiosidades](https://www.formul
5. [OJ Simpson es absuelto tras terminar su libertad condicional](https://www

Estos enlaces te proporcionarán información variada sobre la serie, desde su

FINAL:
 Aquí te presento 5 enlaces relacionados con la historia de Los Simpson:

1. [Caso O. J. Simpson](https://es.wikipedia.org/wiki/Caso_O._J._Simpson)
2. [La Historia de los Simpsons en 2 minutos](https://www.periodistadigital.c
3. [35 años de Los Simpson: un curioso corto de 88 segundos...](https://www.i
4. ['Los Simpson' de aniversario: 25 años de curiosidades](https://www.formul
5. [OJ Simpson es absuelto tras terminar su libertad condicional](https://www

Estos enlaces te proporcionarán información variada sobre la serie, desde su
```

Próximos pasos: otros tipos de agentes

En esta notebook implementamos un agente **ReAct**, donde el modelo:

- razona,
- usa herramientas,
- observa resultados,
- y continúa iterando hasta llegar a una respuesta final.

Sin embargo, existen muchas otras arquitecturas de agentes:

- **Chain-of-Thought (CoT):** razonamiento paso a paso.
- **Plan-and-Execute:** separa planificación y ejecución.
- **Reflection Agents:** el agente revisa y mejora sus propias respuestas.
- **Tree of Thoughts (ToT):** explora múltiples caminos de razonamiento.
- **Multi-Agent Systems:** varios agentes colaboran entre sí.
- **Workflow Agents:** agentes organizados como grafos o flujos de estados.
- **RAG Agents:** agentes conectados a bases de conocimiento externas.

```
1 import pandas as pd
2
```

```
# Assuming 'sample_file.csv' is the file you want to load.
# Replace 'sample_file.csv' with the actual path to your file.
try:
    df_loaded = pd.read_csv('sample_file.csv')
    print('File loaded successfully. First 5 rows:')
    display(df_loaded.head())
except FileNotFoundError:
    print("Error: The file 'sample_file.csv' was not found. Please make
except Exception as e:
    print(f"An error occurred while loading the file: {e}")
```